Name : Vartika Dwivedi
Registration No - 2502050508
Section - D3
DSA Assignment I

Array	Linked List
① Contiguous memory ② May have unused allocated space ③ Time Complexity - Element Access - O(1) Insertion - O(1), O(n) Deletion - O(1), O(n) Searching - O(1), O(n) ④ Implementation is simple	① Non contiguous memor ② Memory allocated dynam as needed ③ Time Complexity - Element Access - O(1), Insertion - O(1), O(n) Deletion - O(1), O(n) Searching - O(1), O(④ Implementation is compl due to pointers.

Array and linked list are linear data structure whe re arranged sequentially.
inked list are suitable for frequent insertion and d ecause elements don't need to be shifted.
g: Music playlist, Dynamic Lists, Stacks, Queues, etc.
Arrays are suitable when fast random access is re ecause elements can be accessed directly using an
g: Student marks, Matrices, Image, Pixels etc.
herefore, linked list is better for frequent insertion / ile array is better for fast random access.

Q.2] → When an array is full and its memory block cannot be extended, insertion is performed as follows :—

① Allocate a larger array dynamically.
② Copy all existing elements into new array.
③ Insert new element at required position.
④ Shift elements if necessary.
⑤ Free old array's memory.
⑥ Display updated array.

If new array has to copy all existing elements, insertion takes O(n) time. Best case can be O(1) only when there is free space and insertion is at the end; but here array is full and must be resized, operation is O(n).

In linked list, new node is created dynamically wherever memory allocation is available. Existing nodes don't need to be moved, only links between nodes are updated.
If insertion position is already known : O(1).
traversal is required to find position : O(n).

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    int n, capacity, pos, value;
    printf("Enter size of array: ");
    scanf("%d", &n);
    capacity = n;
    int *arr = (int*) malloc(capacity * sizeof(int));
    printf("Enter %d elements: \n", n);
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]); }
```

```
printf("Array before insertion: ");
for(int i=0; i<n; i++) {
    printf("%d ", arr[i]); }
printf("\n Enter position (1 to %d): ", n+1);
scanf("%d", &pos);
printf("Enter Elements: ");
scanf("%d", &value);
capacity = capacity * 2;
int *newArr = (int *) malloc(capacity * sizeof(int));
for(int i=0; j=0; i<n+1; i++) {
    if(i == pos-1){
        newArr[i] = value; }
    else {
        newArr[i] = arr[j++]; }
}
free(arr);
arr = newArr;
n++;
printf("Array after insertion: ");
for(int i=0; i<n; i++){
    printf("%d", arr[i]); }
free(arr);
return 0; }
```

Full Array requires larger memory block and copying of elements, whereas a linked list can allocate new node without moving existing elements.

Q.3] → Array stores elements in contiguous memory locations, whereas a linked list stores elements as nodes that can be located at different memory locations.

(A) Array :-
(1) Memory is allocated as a single contiguous block.
(2) Its size is generally fixed.
(3) If more space is required, a larger contiguous array is created and existing elements are copied.
(4) Memory can be released by deleting complete array.
(5) It may waste memory if some allocated locations remain unused.

(B) Linked list :-
(1) Memory is allocated dynamically for each node.
(2) New nodes can be added wherever required without moving existing nodes.
(3) Nodes can use free memory available at different locations.
(4) Individual nodes can be deleted and their memory released.
(5) It provides better flexibility when size of data changes frequently.

Linked list provides better memory utilization when data size changes dynamically, such as in dynamic waiting list. Array is more suitable when size is fixed and fast access is required, such as storing student marks.

∴ Linked list is more flexible for dynamic data, while Array is better for fixed size data requiring fast access.

Q.4] →
```
#include <stdio.h>
int main (){
    int n, i, j = 0, temp;
    printf ("Enter number of elements: ");
    scanf ("%d", &n);
```

```
int arr[n];
printf("Enter Elements: \n");
for(i=0; i<n; i++){
    scanf("%d", &arr[i]); }
for(i=0; i<n; i++){
    if (arr[i] != 0) {
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        j++; }
}
printf("Array after moving zeros to the end: \n");
for(i=0; i<n; i++) {
    printf("%d", arr[i]); }
return 0; }
```

Q.5] →

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int disk;
    struct Node *next; };
struct Stack {
    struct Node* top; };

void push (struct Stack *s, int disk) {
    struct Node* newNode = (struct Node *) malloc (sizeof(
        struct Node));
    newNode -> disk = disk;
    newNode -> next = s -> top;
    s -> top = newNode; }
```

```
int pop (struct Stack *s){
    if (s->top == NULL){
        return -1; }
    struct Node *temp = s->top;
    int disk = temp->disk;
    s->top = s->top->next;
    free (temp);
    return disk; }

int peek (struct Stack *s){
    if (s->top == NULL){
        return -1; }
    return s->top->disk; }

void display (struct Stack *s){
    struct Node *temp = s->top;
    while (temp != NULL){
        printf ("%d", temp->disk);
        temp = temp->next; }
    printf ("\n"); }

void moveDisk (struct Stack *from, struct Stack *to,
               char fromName, char toName){
    int fromTop = peek (from);
    int toTop = peek (to);
    if (fromTop == -1){
        int disk = pop (to);
        push (from, disk);
        printf ("Move disk %d from %c to %c \n", disk,
            fromName, toName); }, fromName); }
    else if (toTop == -1){
```

```
        int disk = pop(from);
        push(to, disk);
        printf("Move disk %d from %c to %c \n",
                disk, fromName, toName); }
    else if(fromTop < toTop) {
        int disk = pop(from);
        push(to, disk);
        printf("Move disk %d from %c to %c \n",
                disk, fromName, toName); }
    else {
        int disk = pop(to);
        push(from, disk);
        printf("Move disk %d from %c to %c \n",
                disk, toName, fromName); }
}

int main() {
    int n;
    printf("Enter number of disks: ");
    scanf("%d", &n);
    struct Stack source = (NULL);
    struct Stack auxiliary = (NULL);
    struct Stack destination = (NULL);
    for(int i = n; i >= 1; i--) {
        push(&source, i); }
    int totalMoves = (1 << n) - 1;
    struct Stack *A = &source;
    struct Stack *B = &auxiliary;
    struct Stack *C = &destination;
    char nameA = 'S';
    char nameB = 'A';
```

```
char nameC = 'D';
if (n % 2 == 0) {
    B = &destination;
    C = &auxiliary;
    nameB = 'D';
    nameC = 'A'; }
for (int i=1; i<= totalMoves; i++) {
    if (i % 3 == 1) {
        moveDisk (A, C, nameA, nameC); }
    else if (i % 3 == 2) {
        moveDisk (A, B, nameA, nameB); }
    else {
        moveDisk (B, C, nameB, nameC); }
}
printf ("\n Final Source Stack: ");
display (&source);
printf ("\n Final Auxiliary Stack: ");
display (&auxiliary);
printf ("Final Destination Stack: ");
display (&destination);
return 0; }
```

Q.6 →

Insertion Sort	Bubble Sort
① Each element is already ~~arranged in ascending order, both~~ its correct position, so only one comparison is needed for each element and no shifting is required.	① Adjacent elements are compared, but since array is already sorted, no swaps are required. In optimized bubble sort, algorithm stops after first pass because no swap occurs.
② Comparisons : $n-1$	② Comparisons : $n-1$
③ Shifts / Movements : 0	③ Swaps : 0

④ Best case : O(n)	④ Best case : O(n)

Both algorithms take O(n) time for an already sorted array. However, Insertion Sort is generally more efficient because it has less overhead and performs well on already or nearly sorted data.

Q.7] → Circular Queue is useful when fixed size queue needs efficient reuse of memory, its main limitation is its fixed capacity. It follows FIFO (First in First out) principle. In a circular queue last position of array is connected to first position, so the empty spaces created by dequeue operations can be reused.

```
#include <stdio.h>
#define MAX 5
int queue [MAX];
int front = -1, rear = -1;

void enqueue (int value) {
   if ((rear +1) % MAX == front) {
      printf ("Queue Full \n");
      return; }
   if (front == -1) {
      front =0; }
   rear = (rear +1) % MAX;
   queue [rear] = value; }

void dequeue () {
   if (front == -1) {
```

```
        printf("Queue Empty \n");
        return; }
    printf("Deleted: %d \n", queue[front]);
    if (front == rear) {
        front = rear = -1; }
    else {
        front = (front+1) % MAX; }

void peek() {
    if (front == -1){
        printf("Queue Empty \n"); }
    else {
        printf("Front: %d \n", queue[front]); }
    }

void display() {
    if (front == -1) {
        printf("Queue Empty \n"); }
        return; }
    int i = front;
    printf("Queue : ");
    while (1) {
        printf("%d", queue[i]);
        if (i == rear){
            break; }
        i = (i+1) % MAX; }
    printf("\n"); }

int main() {
    int choice, value;
    while (1){
```

```
printf("\n 1.Enqueue 2.Dequeue 3.Peek 4.Display
        5. Exit \n");
printf("Enter choice: ");
scanf("%d", &choice);
switch(choice){
    case1: printf("Enter patient value: ");
           scanf("%d", &value);
           enqueue(value);
           data, break;
    case 2: dequeue();
            break;
    case 3: peek();
            break;
    case 4: display();
            break;
    case 5: return 0;
    default: printf("Invalid choice \n"); }
}
}
```

* Advantages :-

① Reuses vacant positions after deletion.
② Better memory utilization than normal linear queue.
③ Enqueue and Dequeue takes O(1) time.

* Limitations :-

① Array has fixed size.
② When queue is full, new patients can't be added until space becomes available.
③ Increasing the size requires larger array.

Q.8] → Data Structure is classified as linear or Non-linear based on how its elements are organized and related to each other. Organization and relationship between elements determine how data structure is traversed and classified.

(A) Linear Data Structure :—
Elements are arranged in sequential order. Each element generally has a single predecessor and successor, except first and last elements. Traversal is usually done sequentially.
Eg: ① Array - Elements stored sequentially & accessed using indexes.

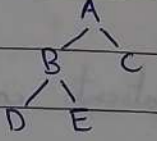
10	20	30	40

② Linked list - Each node points to next node.
10 → 20 → 30 → 40

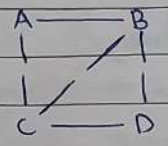
③ Stack - Elements follow LIFO (Last in First Out).
30 ← Top
20
10

④ Queue - Elements follow FIFO (First in First Out).
Front → 10 → 20 → 30 ← Rear

(B) Non-linear Data Structure :—
Elements aren't arranged sequentially. One element can be connected to multiple other elements, creating hierarchical or network like relationships. Traversal can follow different paths.
Eg: ① Tree - Node can have multiple child nodes. It is traversed using methods such as Pre/In/Post order.



② Graph - Vertices can have multiple connections, and traversal is commonly performed using BFS/DFS.



Q.9] → Array is most suitable choice. It stores elements sequentially and provides fast random access using an index. It also allows efficient searching of sorted data using Binary Search (O(log n)).

If insertion and deletion are frequent, a Singly Linked List ~~is preferred because~~ maybe more suitable because elements don't need to be shifted. If data's size changes dynamically, Linked List is preferred because nodes can be added or removed as required.

Stack and Queue aren't suitable because they follow LIFO and FIFO respectively.

Binary Search Tree isn't a good choice because it is more complex and requires extra memory for pointers. For already stored data, array provides simpler storage and efficient access.

Q.10] →
```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left, *right; };

struct Node * createNode (int value) {
    struct Node* newNode = (struct Node*) malloc (sizeof (
        struct Node));
```

```
    newNode -> data = value;
    newNode -> left = new -> right = NULL;
    return newNode; }

struct Node* insert (struct Node* root, int value) {
    if (root == NULL) {
        return createNode (value); }
    if (value < root -> data) {
        root -> left = insert (root -> left, value); }
    else {
        root -> right = insert (root -> right, value); }
    return root; }

void rangeSearch (struct Node* root, int L, int R) {
    if (root == NULL) {
        return; }
    if (root -> data > L) {
        rangeSearch (root -> left, L, R); }
    if (root -> data >= L && root -> data <= R) {
        printf ("%d", root -> data); }
    if (root -> data < R) {
        rangeSearch (root -> right, L, R); }
}

int main() {
    struct Node* root = NULL;
    int n, value, L, R;
    printf ("Enter number of elements: ");
    scanf ("%d", &n);
    printf ("Enter element: \n");
    for (int i=0; i<n; i++) {
```

```
scanf("%d", &value);
root = insert(root, value);}
printf("Enter range L and R: ");
scanf("%d %d", &L, &R);
printf("Elements in range [%d, %d]: ", L, R);
rangeSearch(root, L, R);
return 0;}
```

Q.11] → When values are frequently inserted, deleted and searched while maintaining sorted order, Binary Search Tree is more suitable than an Array.

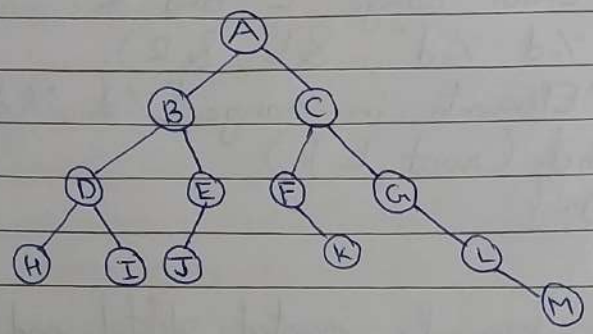
~~In Array, insertion and deletion and searching are frequent. Array is better when~~

In Array insertion and deletion require shifting elements, which takes O(n) time. In a BST, insertion and deletion can be performed without shifting other elements. Inorder traversal of BST gives elements in sorted order

Array	Binary Search Tree
① Contiguous memory.	① Nodes connected by pointers.
② O(log n) for searching.	② O(log n) is average for searching.
③ O(n) for insertion.	③ O(log n) is average for insertion.
④ O(n) for deletion.	④ O(log n) is average for deletion.
⑤ Sorted order maintained by shifting.	⑤ Sorted order maintained by BST property.

BST is more appropriate when insertion, deletion, searching are frequent. Array is better when fast random access is required and modifications are less frequent.

Q.12 → Inorder :— H, D, I, B, J, E, A, F, K, C, G, L, M
Preorder :— A, B, D, H, I, E, J, C, F, K, G, L, M



The Postorder Traversal (Left - ~~Roo~~ Right - Root) is
H, I, D, J, E, B, K, F, M, L, G, C, A.

Q.13 →
```
# include <stdio.h>
# include <stdlib.h>
struct Node {
    int data;
    struct Node * left, * right; };

struct Node * createNode (int value) {
    struct Node * newNode = (struct Node *) malloc (sizeof (
        struct Node));
    newNode → data = value;
    newNode → left = NULL;
    newNode → right = NULL;
    return newNode; }

struct Node * insert (struct Node * root, int value) {
    if (root == NULL) {
        return createNode (value); }
    if (value < root → data) {
        root → left = insert (root → left, value); }
```

```
    else {
        root -> right = insert (root -> right, value); }
    return root; }

void preorder (struct Node * root){
    if (root == NULL){
        return; }
    struct Node * stack [200];
    int top = -1;
    stack [++top] = root;
    while (top != -1){
        struct Node * current = stack [top--];
        printf ("%d ", current -> data);
        if (current -> right != NULL){
            stack [++top] = current -> right; }
        if (current -> left != NULL){
            stack [++top] = current -> left; }
    }
}

int main (){
    struct Node * root = NULL;
    int n, value;
    printf ("Enter number of nodes: ");
    scanf ("%d", &n);
    printf ("Enter elements: ");
    for (int i = 0; i < n; i++){
        scanf ("%d", &value);
        root = insert (root, value); }
    printf ("Preorder traversal: ");
    preorder (root);
```

```
    return 0; }
```

Q.14] →

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node * left, * right; };

struct Node* createNode (int value) {
    struct Node * newNode = (struct Node*) malloc (sizeof (
            struct Node));
    newNode → data = value;
    newNode → left = newNode → right = NULL;
    return newNode; }

struct Node * insert ( struct Node * root, int value) {
    if (root == NULL) {
        return createNode (value); }
    if (value < root→data) {
        root → left = insert (root → left, value). }
    else { root → right = insert (root → right, value); }
    return root; }

int countSmaller (struct Node * root, int k) {
    if (root == NULL) {
        return 0; }
    if (root → data >= k) {
        return countSmaller (root → left, k); }
    return 1 + countSmaller (root → left, k) + countSmaller (
        root → right, k); }
```

```
int main() {
    struct Node * root = NULL;
    int n, value, k;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for(int i=0; i<n; i++){
        scanf("%d", &value);
        root = insert(root, value); }
    printf("Enter k: ");
    scanf("%d", &k);
    printf("Number of elements smaller than %d = %d \n",
            k, countSmaller(root, k));
    return 0; }
```

O/P :- Enter number of elements : 7
Enter elements : 50
30
70
20
40
60
80
Enter k : 50
Number of elements smaller than 50 = 3

* Performance Comparison :-

(A) Balanced BST - Approximately O(log n) to follow relevant path, although this simple counting program may visit additional nodes.

(B) Skewed BST - Can become O(n) because its height can become n-1.

(C) AVL Tree - Remains balanced, so its height is O(log n) and gives more predictable performance.

* Effect of Height :-
Time required depends on height of tree. A smaller height means fewer nodes need to be traversed.

Q.15 →
```
int height (struct Node * root) {
    if (root == NULL) {
        return 0; }
    int left = height (root → left);
    int right = height (root → right);
    return 1 + (left > right ? left : right); }

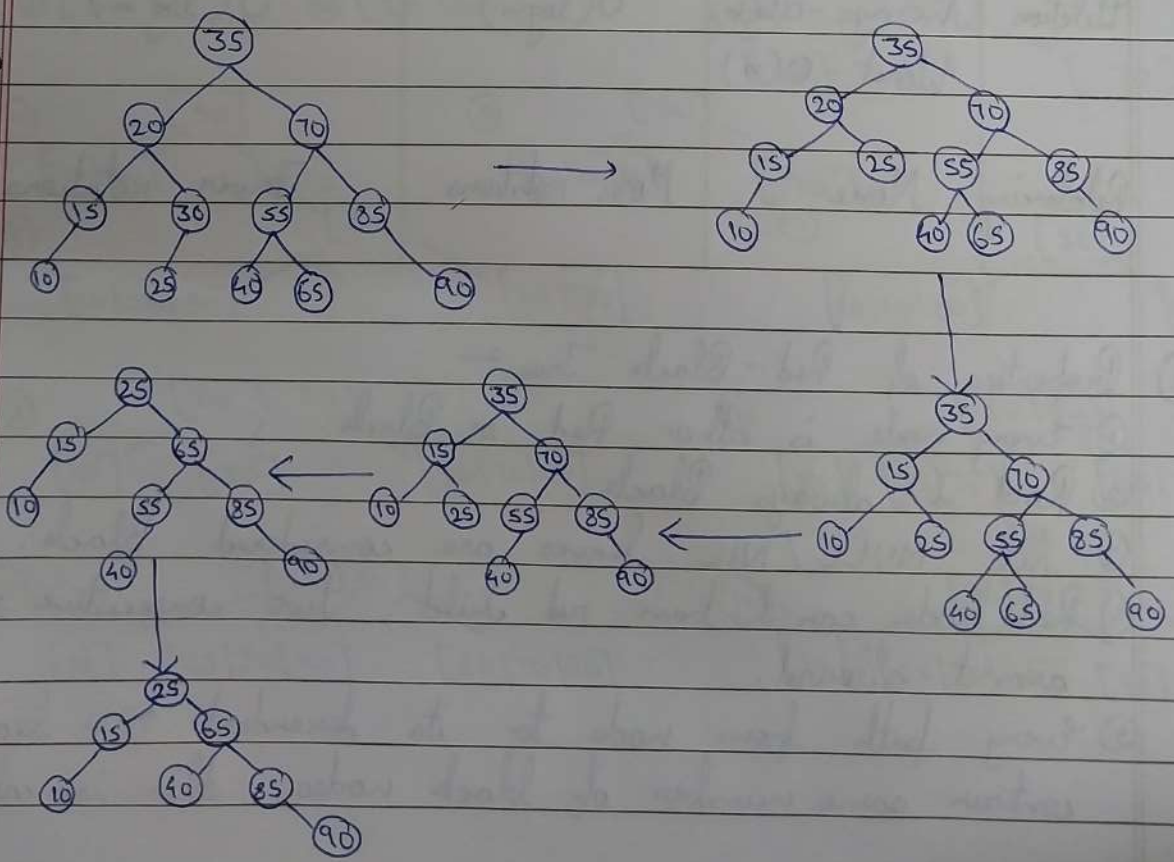
void displayBalanceFactor (struct Node * root) {
    if (root == NULL) {
        return; }
    displayBalanceFactor (root → left);
    int leftHeight = height (root → left);
    int rightHeight = height (root → right);
    int BalanceFactor = leftHeight - rightHeight;
    printf ("Node %d: Balance Factor = %d \n",
            root → data, balanceFactor);
    displayBalanceFactor (root → right); }
```

Q.16 → BST may be balanced or skewed while an AVL Tree always maintains balance using rotations. Performance of searching, insertion and deletion depends mainly on height of tree.

	Balanced (BST)	Skewed (BST)	AVL Tree
Searching	$O(\log n)$	$O(n)$	$O(\log n)$
Insertion	$O(\log n)$	$O(n)$	$O(\log n)$
Deletion	$O(\log n)$	$O(n)$	$O(\log n)$

Balanced BST has smaller height so operations are faster.
Skewed BST can have height close to n, making its operations $O(n)$.
AVL tree maintains its balance after insertion and deletion using rotations, keeping its height $O(\log n)$.
BST can become inefficient when skewed, whereas AVL tree provides more predictable performance because searching, insertion and deletion remain $O(\log n)$.

Q.17 →



Q.13 → (A) Red Black Tree is self balancing ~~Tree~~ binary search tree. It is useful because a normal BST can become skewed and take O(n) time for searching, insertion and deletion.
AVL tree is more strictly balanced, but it may require more rotations after updates. RBT allows a ~~bit~~ little more imbalance so insertion and deletion requires fewer rotations.

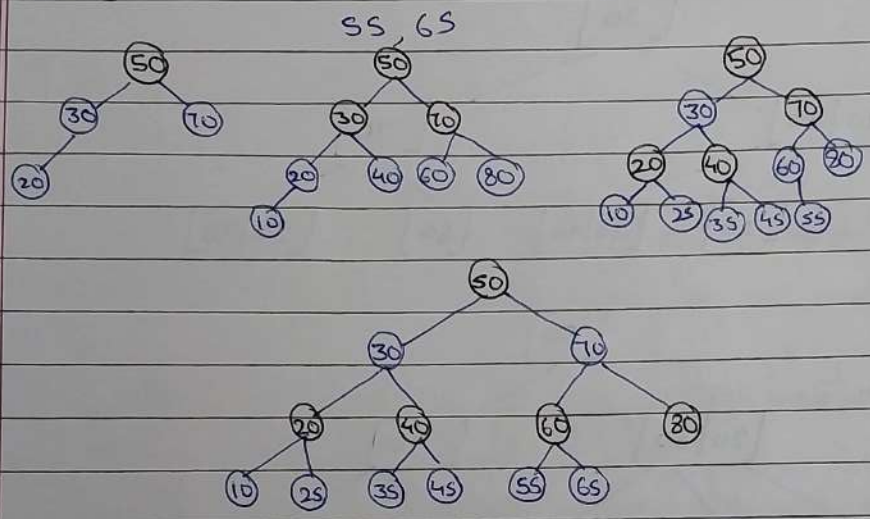
	BST	AVL Tree	RBT
Balance	Not guaranteed	Strictly balanced	Approximately balanced.
Search	Average - O(log n) Worst - O(n)	O(log n)	O(log n)
Insertion	Average - O(log n) Worst - O(n)	O(log n)	O(log n)
Deletion	Average - O(log n) Worst - O(n)	O(log n)	O(log n)
Rebalancing	None	More rotations	Fewer rotations

(B) Properties of Red - Black Tree :-
1. Every node is either Red or Black.
2. Root is always Black.
3. All NULL / NIL leaves are considered Black.
4. Red node can't have red child, two consecutive red nodes aren't allowed.
5. Every path from node to its descendant NIL leaves must contain same number of black nodes, this is called black

height.

⑥ These properties keep tree balanced and ensure O(log n) height.

ⓒ Construct RBT – 50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45, 55, 65



Q.19 → Ⓐ 50, 20, 70, 10, 30, 60, 80, 25, 40, 90, 15, 35, 45

① [50]

② [20|50]

③ [50] → [20], [70]

④ [50] → [10|20], [70]

⑤ [50] → [10|20|30], [70]

⑥ [50] → [10|20|30], [60|70]

⑦ [50] → [10|20|30], [60|70|80]

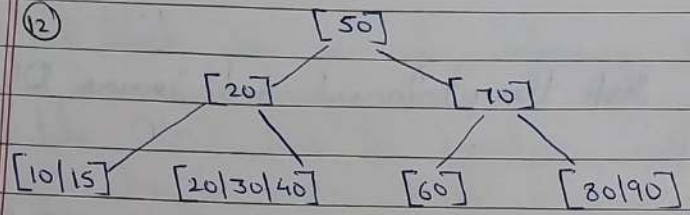
⑧ [20|50] → [10], [30], [60|70|80]

⑨ [20|50] → [10], [25|30], [60|70|80]

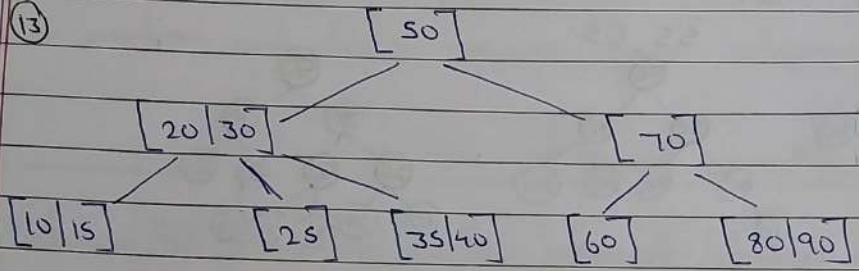
⑩ [20|50] → [10], [25|30|40], [60|70|80]

⑪ [20|50|70] → [10], [25|30|40], [60], [80|90]

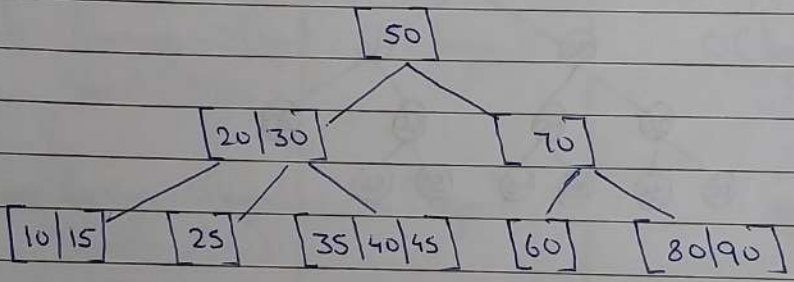
(12)



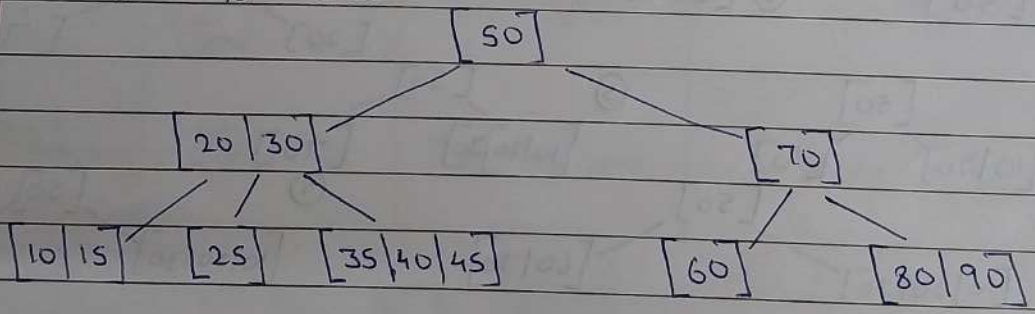
(13)



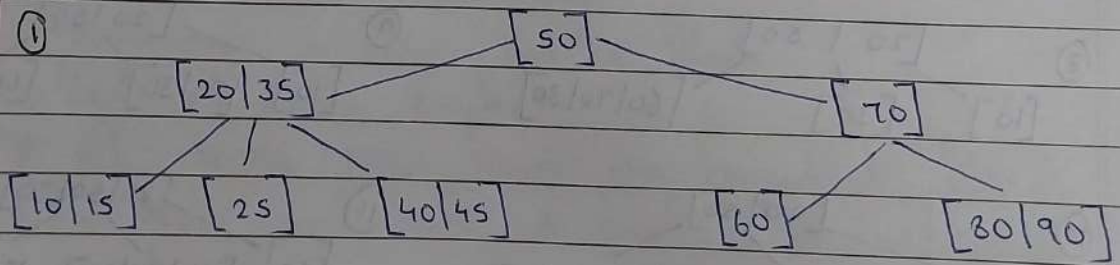
(14)



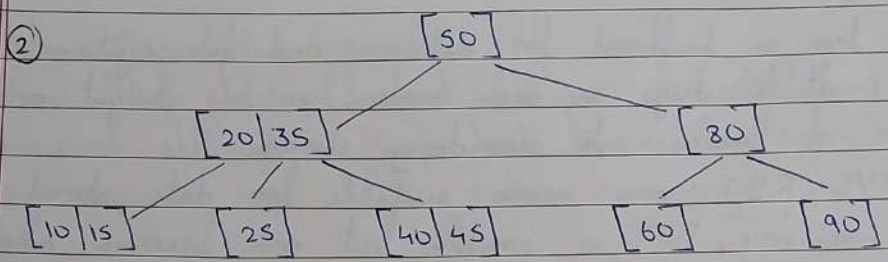
(B) → Final B - Tree :—



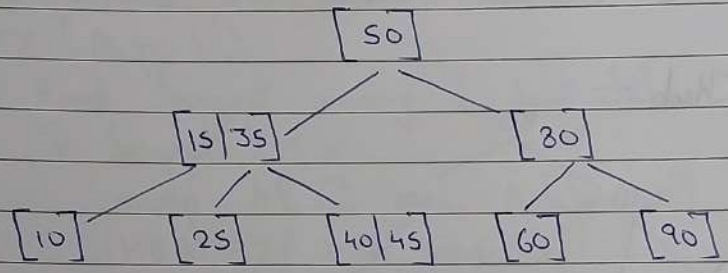
(C) → (1)



(2)



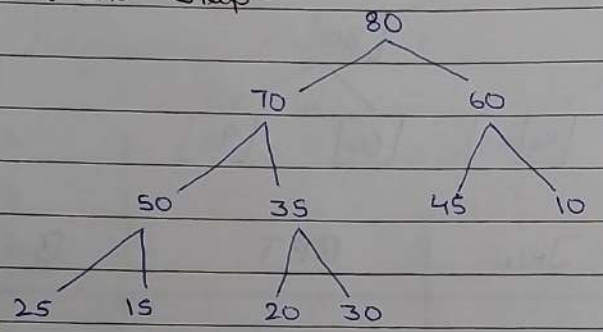
(3)



Q.20] →	BST	AVL Tree	RBT	B-Tree
Structure	Binary tree.	Balanced binary tree.	Balanced binary tree.	Multiway tree.
Height	O(n)	O(log n)	O(log n)	O(log n)
Searching	O(n)	O(log n)	O(log n)	O(log n)
Insertion	O(n)	O(log n)	O(log n)	O(log n)
Deletion	O(n)	O(log n)	O(log n)	O(log n)
Memory	Nodes with pointers.	Nodes with pointers.	Nodes with pointers and colors.	Multiple keys per node.
	No	Rotations	Rotations and rebalancing	Splitting and merging.

A B - tree is preffered for databases and file systems because it stores multiple keys in each node, has low height and reduces number of accesses to secondary storage.
BST, AVL, RBT are more suitable for data stored mainly in main memory. BST is simple but can become skewed, while AVL and RBT maintain balance and provide better performance.

Q.21. (A) → Max Heap :-



* Heap Sort :-

30, 70, 60, 50, 35, 45, 10, 25, 15, 20, 30
70, 50, 60, 25, 35, 45, 10, 30, 15, 20 | 80
60, 50, 45, 30, 35, 20, 10, 25, 15 | 70, 80
50, 35, 45, 30, 35, 20, 10, 25 | 60, 70, 80
45, 35, 25, 30, 15, 20, 10 | 50, 60, 70, 80
35, 30, 25, 10, 15, 20 | 45, 50, 60, 70, 80
30, 20, 25, 10, 15 | 35, 45, 50, 60, 70, 80
25, 20, 15, 10 | 30, 35, 45, 50, 60, 70, 80
20, 10, 15 | 25, 30, 35, 45, 50, 60, 70, 80
15, 10 | 20, 25, 30, 35, 45, 50, 60, 70, 80
10 | 15, 20, 25, 30, 35, 45, 50, 60, 70, 80

∴ Final Sorted Array is | 10 | 15 | 20 | 25 | 30 | 35 | 45 | 50 | 60 | 70 | 80 |

* Why Max Heap :-
Max Heap keeps largest element at root so it can be removed efficiently. Heap Sort has $O(n \log n)$ time complexity and requires very little extra memory.

* Recursive Heapify Algorithm :-
```
HEAPIFY (A, n, i)
    Find largest among i, left child, right child
    if largest != i
        swap A[i] and A[largest]
        HEAPIFY(A, n, largest)
```

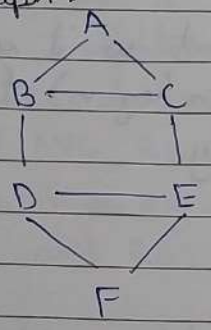
* Non-recursive Heapify Algorithm :-
```
HEAPIFY (A, n, i)
    Repeat :
        Find largest among i, left child, right child
        if largest == i, stop
        swap A[i] and A[largest]
        i = largest
```

* Comparison :-

	Best Case	Average Case	Worst Case
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$

Q.22] → Vertices - A, B, C, D, E, F Edges : (A,B), (A,C), (B,C), (B,D), (C,E), (D,E), (D,F), (E,F)

* Graph :-



* Adjacency Matrix :-

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	1	0	0
C	1	1	0	0	1	0
D	0	1	0	0	1	1
E	0	0	1	1	0	1
F	0	0	0	1	1	0

* Adjacency list :-

A → B → C

B → A → C → D

C → A → B → E

D → B → E → F

E → C → D → F

F → D → E

* C Program :-

```
#include <stdio.h>
#define V 6
char vertices [] = {'A', 'B', 'C', 'D', 'E', 'F'};
int main () {
  int matrix [v][v] = {0};
```

```
int edges[][2] = {
    {0,1}, {0,2}, {1,3},
    {2,4}, {3,4}, {3,5}, {4,5} };
int E = 8;
for (int i=0; i<E; i++) {
    int u = edges[i][0];
    int v = edges[i][1];
    matrix[u][v] = 1;
    matrix[v][u] = 1; }
printf("Adjacency Matrix : \n");
for (int i=0; i<V; i++)
    printf("%c", vertices[i]);
printf("\n");
for (int i=0; i<V; i++)
    printf("%c", vertices[i]);
    for (int j=0; j<V; j++)
        printf("%d", matrix[i][j]);
    printf("\n"); }
printf("\n Adjacency List: \n");
for (int i=0; i<V; i++) {
    printf("%c -> ", vertices[i]);
    for (int j=0; j<V; j++) {
        if (matrix[i][j])
            printf("%c ", vertices[j]); }
    printf("\n"); }
return 0; }
```

* Comparison :-

Operation	Adjacency Matrix	Adjacency List
Memory	$O(V^2)$	$O(V+E)$

Check edge	O(1)	O(v)
Add edge	O(1)	O(1)
Remove edge	O(1)	O(v)
Traverse neighbours	O(v)	O(degree of vertex)

* Suitable Representation :-
For a sparse graph, an Adjacency list is better because it stores only existing edges and uses O(V+E) memory.

For a dense graph, an Adjacency Matrix is more suitable because most possible edges exist and checking an edge takes O(1) time.

Q.23] → (A) A → F → D → J
(B) J → K